CAREERPILOT AI

Revised Synopsis, Software Requirements Specification and Implementation Roadmap

A responsive career preparation platform for students and placement teams

BACHELOR OF COMPUTER APPLICATIONS
FINAL YEAR MAJOR PROJECT

Revised 18 October 2026

Planning assumption: one institution, English interface, responsive web app with installable PWA support. Native Android/iOS apps are outside the first release.

How to use this document

This document consolidates a revised synopsis, implementation-oriented SRS, and phased roadmap. It turns the original project idea into testable requirements and aligns the laptop and mobile experience. Quality targets are proposed, not results already measured; confirm them with your project guide and college rubric.

The timeline assumes one student working about 10–15 hours per week. Adjust it to your actual schedule, API access, hosting, and project submission date.

Contents

- 1. Revised project synopsis
- 2. Product scope and user roles
- 3. Software Requirements Specification
- 4. Data, architecture and AI design
- 5. Verification and acceptance plan
- 6. Implementation roadmap
- 7. Additional changes and risks
- 8. Definition of done

1. Revised Project Synopsis

Project title

CareerPilot AI: An AI-Assisted Career and Placement Preparation Platform for College Students

Abstract

CareerPilot AI is a responsive web application that helps college students organize placement preparation in one place. Students can build or upload a resume, review AI-assisted feedback, compare skills with a target role, follow learning recommendations, practice interviews, complete coding and aptitude exercises, and track progress. Placement officers can review cohort-level readiness, identify common skill gaps, manage student records, and publish notifications. The app supports laptop and mobile browsers and can be installed as a Progressive Web App. It uses React and Tailwind CSS, a Node.js/Express API, a Python/FastAPI AI service, and PostgreSQL. AI feedback is advisory; readiness uses a disclosed formula rather than an unexplained model output.

Problem statement

Students often use disconnected tools for resumes, learning, interviews, and practice, making progress difficult to manage. Placement teams also need a consistent view of preparation and common skill gaps. One role-based platform can organize these activities and provide useful summaries while protecting student information.

Aim and objectives

Aim: design, implement, and evaluate a secure, responsive platform for placement preparation and authorized cohort progress tracking.

- Provide sign-up, authentication, profiles, and separate student, placement officer, and administrator permissions.
- Support resume creation/upload, structured analysis, and editable AI-assisted suggestions.
- Compare student skills with target roles and recommend relevant learning resources.
- Provide mock interviews, coding and aptitude practice, saved attempts, and feedback.
- Show a transparent readiness summary and student progress dashboard.
- Give authorized placement officers cohort reports and in-app notifications.
- Verify laptop/mobile usability, core accessibility, security, and end-to-end workflows.

Scope

First release	Future scope
One institution; English; student, placement officer, administrator; one deployment.	Separate native mobile apps, multiple colleges, multilingual UI.
Responsive laptop/mobile web app and installable PWA; online access for AI/server features.	Offline AI, voice/video interviews, live collaborative sessions.
Resume builder/upload and analysis, skill gaps, career/learning recommendations, interview, coding/aptitude practice, readiness, dashboards, reports, in-app notices.	LinkedIn/GitHub, job-board/ATS integrations, job applications, fine-tuned models.
One configured AI provider behind an adapter; useful error/fallback states.	Provider choice exposed to end users.

Expected outcomes

A working demonstrable application; version-controlled source; database schema and seed data; reproducible local or hosted demo; requirements/design documentation; test evidence; and a user guide. The system supports preparation and tracking; it does not guarantee employment.

Proposed technology stack

Layer	Technology / responsibility
Frontend	React, Tailwind CSS, responsive UI, PWA manifest/installability.

Layer	Technology / responsibility
Application API	Node.js and Express: authentication, authorization, validation, workflows, reports.
AI service	Python and FastAPI: resume analysis, recommendations, interview feedback behind a provider adapter.
Database	PostgreSQL: accounts, profiles, resources, attempts, scores, notices, audit events.
Authentication	JWT-based sessions, secure cookie handling, password hashing, authorization and logout/revocation.
External AI	Choose OpenAI or Gemini for the first release; keep keys on the server.

2. Product Scope and User Roles

Primary workflows

Student: register/sign in → complete profile → build/upload resume → review analysis → select target role → review skill gaps/resources → practice interview, coding, and aptitude → review readiness/progress → respond to notices.

Placement officer: sign in → view assigned cohort metrics → search/filter students → view permitted summaries → publish cohort notices. Administrator: manage accounts, roles, catalogs, and configuration; administrative access should be limited and auditable.

Role and access matrix

Capability	Student	Placement officer	Administrator
Edit own profile/resume	Yes	No; view only if authorized	Support access only when needed
Use preparation features	Yes	No	No by default
View own attempts/readiness	Yes	No	No by default
View cohort progress	No	Assigned cohort only	System-level when needed
Publish placement notices	No	Assigned cohort	All cohorts
Manage roles/configuration	No	No	Yes

Raw resumes and detailed AI feedback must not be exposed to officers by default. Define institutional approval and student notice before enabling this access.

3. Software Requirements Specification

3.1 Purpose and product boundary

The first release is a single-institution responsive web app/PWA. React communicates with the Express API over HTTPS. The API owns authorization, business rules, and persistence. It calls PostgreSQL and the Python AI service; the AI service calls the configured provider. The AI service must not independently grant access to student data.

3.2 Functional requirements

ID	Requirement	Acceptance evidence
FR-01 Identity	Register, sign in/out, and recover access; never store plain-text passwords.	Valid user reaches authorized routes; invalid sign-in is rejected without revealing account existence.
FR-02 Roles	Enforce Student, Placement Officer, and Administrator permissions in UI and API.	Direct unauthorized API requests fail without disclosing protected data.
FR-03 Profile	Student can maintain education, cohort, target role, and preferences.	Saved values persist and validate across sessions.
FR-04 Resume	Create a structured resume; upload supported PDF/DOCX under a configured size; view, replace, delete own versions.	Reject unsupported/oversized files clearly; prevent cross-student file access.
FR-05 Analysis	Analyze own resume and show extracted skills, section feedback, explained score, suggestions.	Analysis links to resume version; failures/pending state are visible; do not invent credentials.
FR-06 Skill gaps	Choose a target role and compare student skills/proficiency with role requirements.	Show matched/missing skills and the catalog/version used.
FR-07 Resources	Show curated resources mapped to gaps and allow progress tracking.	Each has title, skill, source/link, and unavailable-link state.
FR-08 Interviews	Start a session, answer prompts, submit, and review saved feedback.	Progress persists; interrupted sessions are resumable or marked incomplete.
FR-09 Coding	Browse problems, submit answers, and review result/test feedback.	Attempts persist. Executable code runs only in isolated, resource-limited runner; otherwise use non-executable exercises.

ID	Requirement	Acceptance evidence
FR-10 Aptitude	Take configured tests and review score, answers, and history.	Score against stored answer key; attempt remains available to its owner.
FR-11 Readiness	Calculate summary from disclosed categories and available practice data.	Show category values, formula version, and insufficient-data state; never imply placement probability.
FR-12 Student dashboard	Show progress, recommendations, upcoming activities, and notices.	Actions link to functioning screens and reflect saved data.
FR-13 Officer dashboard	Show cohort metrics, search/filter, and permitted performance summaries.	Changing URL/request parameters cannot access another cohort.
FR-14 Notifications	Officer/admin creates, edits, publishes, archives cohort-targeted in-app notices.	Only target cohort sees the notice; read state is stored.
FR-15 Audit/deletion	Log security-relevant administrative actions and support deactivation/deletion per policy.	Role/notice changes are attributable; deletion and retention behavior is documented.
FR-16 Responsive/PWA	Make core flows usable on laptop/mobile browsers; support install where PWA is available.	At agreed viewports, no hidden primary actions or horizontal page scrolling.

3.3 Proposed quality requirements

ID	Proposed target
NFR-01 Responsive	Check 360×800, 390×844, 768×1024, and 1366×768; reflow without losing core actions.
NFR-02 Performance	Target common non-AI API reads under 2.5s at p95 on demo host; AI calls show progress, timeout, and error state.
NFR-03 Security	HTTPS in deployment, password hashing, server-side authorization, validation, safe file handling, rate limits, server-side secrets.
NFR-04 Privacy	Collect minimum data; explain provider processing; synthetic demo data; define retention/deletion before real student data.
NFR-05 Accessibility	Keyboard use, visible focus, semantic labels, readable contrast, text zoom, clear validation messages.
NFR-06 Reliability	Failures do not corrupt attempts; AI failure does not erase draft answers.
NFR-07 Maintainability	Separate frontend/API/AI/persistence; document setup, config, migrations, and deployment.
NFR-08 Compatibility	Verify current Chrome/Edge/Firefox desktop and mobile Chrome/Safari where available; record versions.
NFR-09 Integrity	Use constraints/transactions; link analyses and scores to source versions.

Targets are for validation on the selected demo environment, not performance guarantees independent of hosting or provider latency.

4. Data, Architecture and AI Design

4.1 Components

Component	Responsibility and boundary
React + Tailwind PWA	Responsive screens, navigation, forms, dashboards, install metadata; no provider keys; client checks are not authorization.
Express API	Authentication, role checks, validation, business rules, transactions, files, reports; authoritative security boundary.
FastAPI AI service	Task-specific processing and provider adapter; minimal task data, authenticated service calls, validated structured output.
PostgreSQL	Persistent data with migrations, foreign keys, ownership constraints, and suitable backups.
External provider	One configured AI provider; document data handling, cost, quotas, availability, and rate limits.

Use a three-layer drawing only as a labeled high-level view. The detailed view should consistently show frontend, API, AI service, database, and external provider.

4.2 Proposed logical data model

Entity	Purpose / relationship
USER	Account, role, status; owns student or staff profile.
STUDENT_PROFILE; STAFF_PROFILE; COHORT	Education/target role; officer assignment and cohort scope.
RESUME; RESUME_VERSION; RESUME_ANALYSIS	Owned versions; each analysis references a specific version and stores status/score explanation.
SKILL; STUDENT_SKILL; ROLE_SKILL; LEARNING_RESOURCE	Skill catalog, student proficiency, role requirements, curated resources.
INTERVIEW_SESSION; QUESTION; RESPONSE	Session, prompt, response, and feedback.
CODING_PROBLEM; SUBMISSION	Problem catalog and attempts; runner result metadata where applicable.
APTITUDE_TEST; QUESTION; ATTEMPT	Test catalog, answers, and scored attempts.

Entity	Purpose / relationship
READINESS_SNAPSHOT	Category values, formula version, source references, calculation time.
NOTIFICATION; DELIVERY	Target cohort, author, publication dates, read state.
AUDIT_EVENT	Security-relevant actions; do not store secrets or full resume contents.

After migrations exist, redraw the ERD from the actual schema. Clarify cardinalities for user/profile, resume/analysis, student/attempt, skill/role/resource, and officer/cohort.

4.3 Readiness formula

Proposed starting weights: Resume 25%, Skill Match 25%, Interview 20%, Coding 15%, Aptitude 15%. Get guide approval before implementation. Version the formula; show category contributions and last calculation. If the minimum evidence threshold is not met, show 'not enough activity yet'. Do not label this score as placement probability.

4.4 AI safeguards

- Validate structured provider output before displaying or saving it; treat resumes and answers as untrusted input.
- Show suggestions for student review; never invent achievements, skills, or credentials.
- Provide progress, retry, timeout, provider failure, and quota states without losing drafts.
- Record task/model/status metadata for debugging; avoid logging full resume text or secrets.
- Use synthetic data in development. Inform users and obtain institutional approval before sending real resume data to an external provider.

5. Verification and Acceptance Plan

Test area	Evidence
Unit	Validation, authorization, readiness formula/bounds, parsers, and state logic.
Integration	API and test DB cover ownership, uploads, attempts, notices, and role-scoped reports.
AI	Success, malformed output, timeout, quota, and retry paths using test stub and configured provider.
End-to-end	Student preparation journey; officer sees assigned cohort and publishes notice.
Responsive	Screenshots/checklist at target viewports; navigation, forms, tables, dialogs, keyboard.
Security	Role bypass, cross-user/file access, invalid/large files, rate limits, no secrets in client/repo.
Usability	3–5 representative users/classmates attempt tasks; record issues and changes; do not claim broad research.

Acceptance journey

1. Student registers, completes profile, and reaches dashboard.
2. Student creates/uploads a resume, requests analysis, reviews feedback, and keeps original content.
3. Student selects role, reviews gaps/resources, and records interview, aptitude, and coding practice.
4. Dashboard explains readiness categories and initially shows insufficient data; later score is reproducible.
5. Officer sees only assigned cohort, reviews permitted metrics, and publishes a target notice.
6. The journey works on mobile browser without clipped controls, hidden actions, or horizontal scrolling.

6. Implementation Roadmap

Indicative duration: 12–14 weeks at 10–15 hours/week. Build vertical slices, demonstrate each week, and keep responsive checks in every phase.

Phase	Estimate	Work	Exit gate
0. Scope approval	2–3 days	Confirm features, roles, PWA scope, AI provider, score weights, privacy and rubric; update diagrams and acceptance checklist.	Guide-approved scope; decisions logged.
1. Foundation	3–4 days	Set up Git, React, Express, FastAPI, PostgreSQL, env examples, health checks, formatting and setup guide.	All components run locally; secrets excluded.
2. Responsive shell	1 week	Navigation, role shells, forms, dashboard patterns, loading/empty/error states, PWA manifest/install.	Main routes work at phone and laptop sizes.
3. Data/auth/roles	1–1.5 weeks	Migrations, synthetic seed data, auth/session, profiles, role checks, cohort scoping.	Cross-user/cohort access checks pass.
4. Resume + AI	1–1.5 weeks	Builder/upload, validation, safe storage, versions, analysis, structured feedback, recovery states.	Student can create/analyze/revisit/delete own resume.

Phase	Estimate	Work	Exit gate
5. Career/skills/resources	1 week	Role/skill catalogs, proficiency, gap comparison, curated resources, progress.	Explainable matched/missing skill results.
6. Interview	1 week	Templates, prompts, saved responses, AI feedback, retry/resume UI.	Session persists and can be reviewed.
7. Coding/aptitude	1–1.5 weeks	Question catalog, attempts, results, history; isolated runner for executable code or use non-executable format.	Attempts persist; no unsafe in-process code execution.
8. Readiness/dashboards	1 week	Guide-approved formula/version, missing-data state, student view, cohort metrics/search/filter.	Score is reproducible; officer data scoped.
9. Notices/admin	3–5 days	Cohort notices, publish/archive/read states, minimal account/content admin, audit events.	Only target cohort receives notice.
10. QA/security/responsive	1–1.5 weeks	Unit/integration/e2e; permissions/files; keyboard/accessibility; viewport, performance, backups.	Critical flows pass; blockers resolved.
11. Deployment/submission	3–5 days	Deploy, configure HTTPS/secrets, seed synthetic demo, user guide, screenshots, test summary, demo script.	Reproducible setup; demo and submission ready.

Recommended implementation order

Authentication/profile → resume and analysis → skill gaps/resources → practice modules → readiness/dashboard → officer reports/notifications. Deploy a skeleton early so database, secrets, and provider connectivity are not last-minute surprises.

Weekly rhythm

- Choose 3–5 measurable tasks and a demo outcome each week.
- Keep the main branch runnable and commit working slices at each milestone.
- Track API access, hosting, file storage, and code-runner blockers early.
- Save screenshots, test evidence, and scope decisions; show the build to your guide weekly.

7. Additional Changes and Risks

Other materials to align

- Deck: add a title/takeaway to the use-case slide; label three-layer architecture as high-level; make detailed architecture match the SRS.
- Coverage: every feature must appear in requirements, a screen/flow, data model where needed, and an acceptance check. Include Resume Builder, coding, aptitude, officer reports, notice targeting.
- ERD: redraw from migrations and remove entities/attributes not used by the app.
- Wireframes: add mobile states for navigation, resume, practice, and officer tables; include empty/loading/error/success.
- Use consistent terms: Student, Placement Officer, Administrator, Readiness Snapshot, Resume Analysis, Learning Resource.

Risks and mitigations

Risk	Mitigation / decision
Scope exceeds time	Lock release scope; finish a working MVP slice first; move lower-priority items to future scope.
AI access/cost/provider change	Choose one demo provider; use adapter, quotas, test stub, and documented dependency.
Sensitive resumes	Use synthetic data; minimize transmission; define consent, retention, deletion, officer access.
Unsafe code execution	Never run arbitrary submissions inside API/AI/database process; use isolated runner with resource/network limits.
Misleading readiness score	Publish formula/category values, data coverage, and version; avoid placement probability claims.
Mobile deferred	Use responsive components continuously and test an actual phone if available.
Late deployment	Deploy skeleton in foundation phase; verify secrets/database/provider before feature freeze.

Decisions for project guide

- Approve feature set and executable versus non-executable coding practice.
- Choose AI provider and confirm external resume processing is permitted.
- Approve readiness weights/minimum evidence and cohort report fields.
- Confirm hosting and whether PWA meets the mobile requirement.
- Confirm academic format, required diagrams, test evidence, and final submission sections.

8. Definition of Done

- Approved scope matches synopsis, SRS, screens, schema, and presentation.
- Student preparation and officer cohort workflows work end to end.
- Primary flows work on laptop and mobile; PWA installation is checked where supported.
- API enforces roles/ownership; cross-user access is tested; provider secrets stay server-side.
- AI/file failures are recoverable; privacy and data handling are documented.
- Readiness is transparent/versioned/reproducible; code judging never runs in the app process.
- Critical unit, integration, end-to-end, responsive, and security checks have evidence; high-severity issues are fixed.
- Setup/deployment guide, synthetic demo accounts, user guide, test summary, updated diagrams, and presentation are ready.

This roadmap defines work needed for a complete first release; it does not claim application features are already implemented.